For nearly all modern applications, **AES-256-GCM** (Galois/Counter Mode) is the standard implementation choice because it provides authenticated encryption (AEAD), guaranteeing both data confidentiality and integrity against tampering without requiring a separate HMAC.
Because your exact tech stack and use case aren't specified yet, below are production-ready implementations for the two most common environments—encrypting application data (such as database fields, tokens, or payloads) in **Python** and **Node.js**—followed by a few questions to tailor this to your exact system.

Python Implementation (using cryptography)

Install the standard library via pip install cryptography. This implementation generates a random 12-byte nonce per encryption and packs the nonce + ciphertext + auth_tag into a single Base64 string for easy storage.

```
import os
import base64
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography.hazmat.primitives.kdf.scrypt import Scrypt

# 1. Key Generation (Store this 32-byte key in a KMS, Vault, or
environment variable)
def generate_key() -> bytes:
    return AESGCM.generate_key(bit_length=256)

# Optional: Derive a 32-byte key from a user password using Scrypt
def derive_key_from_password(password: str, salt: bytes) -> bytes:
    kdf = Scrypt(salt=salt, length=32, n=2**14, r=8, p=1)
    return kdf.derive(password.encode("utf-8"))

# 2. Encrypt (Returns Base64 encoded payload containing Nonce +
Ciphertext + Tag)
def encrypt(plaintext: str, key: bytes, associated_data: bytes = None)
-> str:
    aesgcm = AESGCM(key)
    nonce = os.urandom(12)  # 96-bit unique nonce required for GCM
    ciphertext = aesgcm.encrypt(nonce, plaintext.encode("utf-8"),
associated_data)
    return base64.urlsafe_b64encode(nonce +
ciphertext).decode("utf-8")

# 3. Decrypt (Verifies authentication tag automatically before
returning plaintext)
def decrypt(token: str, key: bytes, associated_data: bytes = None) ->
str:
    raw = base64.urlsafe_b64decode(token.encode("utf-8"))
    nonce, ciphertext = raw[:12], raw[12:]
    aesgcm = AESGCM(key)
    plaintext = aesgcm.decrypt(nonce, ciphertext, associated_data)
    return plaintext.decode("utf-8")

# Example Usage
```

```
if __name__ == "__main__":
    secret_key = generate_key()
    # Optional context binding (e.g., user ID or record ID) so
ciphertext cannot be moved to another record
    context = b"user_id:9481"

    encrypted = encrypt("Sensitive medical or financial record",
secret_key, associated_data=context)
    print(f"Encrypted: {encrypted}")

    decrypted = decrypt(encrypted, secret_key,
associated_data=context)
    print(f"Decrypted: {decrypted}")
```

Node.js Implementation (using built-in node:crypto)

No external dependencies are required. Node.js manages the 16-byte authentication tag separately from the ciphertext, so both the 12-byte IV (nonce) and the 16-byte tag are packed alongside the ciphertext.

```
import crypto from 'node:crypto';

const ALGORITHM = 'aes-256-gcm';
const IV_LENGTH = 12;  // 96-bit nonce for GCM
const TAG_LENGTH = 16; // 128-bit authentication tag

// 1. Generate a random 32-byte (256-bit) master key
export function generateKey() {
  return crypto.randomBytes(32);
}

// 2. Encrypt string into a combined Base64 payload (IV + AuthTag +
Ciphertext)
export function encrypt(plaintext, key, aad = null) {
  const iv = crypto.randomBytes(IV_LENGTH);
  const cipher = crypto.createCipheriv(ALGORITHM, key, iv, {
authTagLength: TAG_LENGTH });

  if (aad) {
    cipher.setAAD(Buffer.from(aad));
  }

  const encrypted = Buffer.concat([cipher.update(plaintext, 'utf8'),
cipher.final()]);
  const tag = cipher.getAuthTag();

  return Buffer.concat([iv, tag, encrypted]).toString('base64url');
}

// 3. Decrypt and verify authenticity
```

```
export function decrypt(payload, key, aad = null) {
  const buffer = Buffer.from(payload, 'base64url');
  const iv = buffer.subarray(0, IV_LENGTH);
  const tag = buffer.subarray(IV_LENGTH, IV_LENGTH + TAG_LENGTH);
  const ciphertext = buffer.subarray(IV_LENGTH + TAG_LENGTH);

  const decipher = crypto.createDecipheriv(ALGORITHM, key, iv, {
authTagLength: TAG_LENGTH });
  decipher.setAuthTag(tag);

  if (aad) {
    decipher.setAAD(Buffer.from(aad));
  }

  const decrypted = Buffer.concat([decipher.update(ciphertext),
decipher.final()]);
  return decrypted.toString('utf8');
}
```

Critical Implementation Safeguards

- **Never reuse a Nonce/IV:** Reusing a 12-byte nonce with the same key in AES-GCM catastrophically breaks confidentiality and exposes the authentication key. Always generate a fresh random nonce via a cryptographically secure RNG (os.urandom or crypto.randomBytes) for every encryption call.
- **Bind context with AAD:** Pass an immutable identifier (like a database row ID or user ID) as Associated Authenticated Data (associated_data / aad) so an attacker cannot copy an encrypted field from one user's record and paste it into their own.
- **Large files require chunked streaming:** AES-GCM requires verifying the authentication tag before releasing plaintext. If encrypting files larger than a few megabytes, encrypt in sequential authenticated chunks (e.g., 64 KB blocks with an incrementing counter in the nonce/AAD) to avoid loading the entire file into memory.

To tailor this to your specific application, share a few details:

1. **Language & Environment:** What language, framework, or hardware platform (e.g., C++, Go, Rust, Java, WebCrypto in browser, embedded MCU) are you targeting?
2. **Data Profile:** Are you encrypting small database fields/API payloads, streaming large files, or encrypting local disk storage?
3. **Key Source:** Will the 256-bit key come from a cloud KMS/environment variable, or does it need to be derived from a user-supplied password?